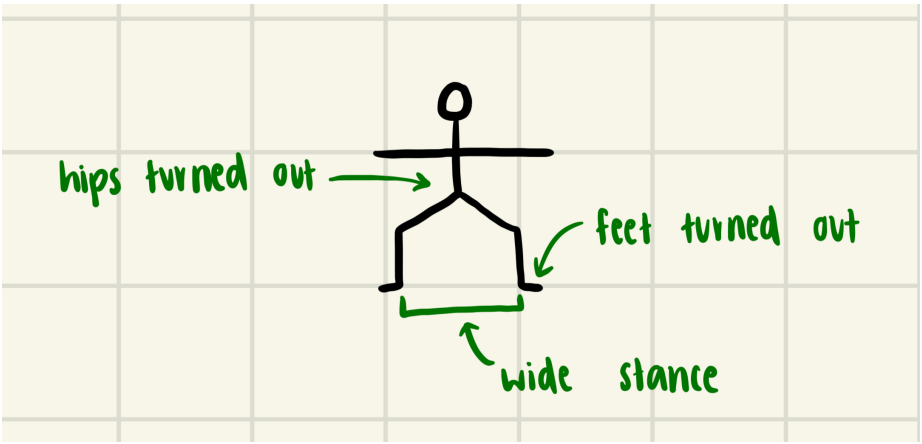
ME133a Final Report

Claire Hays, Rupali Batta, Jana Woo

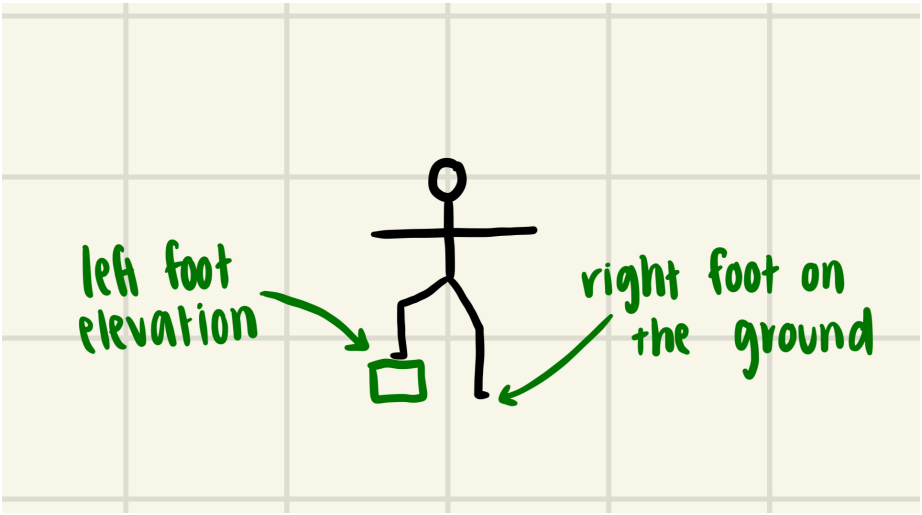
December 2024

1 Overall Description

Our overall goal was to have Atlas perform a jump with sufficient generalization to be capable of adjusting to different starting conditions, etc. This would require squatting down and jumping, using gravity. We only moved the legs for this motion, with the arms maintained in a horizontal position without being moved during the motion. We were able to achieve different starting positions for the hips and feet. For example, Atlas is able to jump like a frog.



Atlas can also jump from when its legs start from any elevation, even if one leg is elevated differently than the other. This can be if one of Atlas' legs are on a block while the other is on the ground. In our final project, we demonstrate this, utilizing a marker in rviz.



Lastly, Atlas can perform a jump with twists regardless of any leg starting

position. In the video, Atlas does a half twist and a full twist. Additionally, all these capabilities are captured in the video linked at the end of the report.

2 System Description

We utilized two 6 DOF legs for a total of 12 DOFs. The workspace is essentially a cylinder with the height being the height of the jump and the radius being then the length of one of Atlas' arms plus half the width of its torso. This is because we don't have Atlas jumping with its legs longer than its arm span and then when Atlas jumps and spins it creates a cylindrical workspace.

As a jump is composed of multiple components, we decided to tackle this project in stages. The initial stage was to have Atlas squat. We approached this by having Atlas' feet fixed to the ground and to simply move the pelvis up and down and have the legs bend to accommodate this movement through inverse kinematics calculations. But, when we evolved to a jump, we changed to a world frame to have an ungrounded urdf, allowing Atlas to move its feet off the ground. As mentioned earlier, the arms remain horizontal. Thus, we incorporated some joint constraints to keep the arms horizontal (although for coding purposes this mostly just meant not sending joint positions for the arms after initialization). We initially faced an issue where the knee joints would bend backwards and forwards, alternating in and out during a squat. This was because our original inverse kinematics calculations did not involve a secondary task to keep the knees bending in the correct direction, so we were calculating the positions of the joints purely based on getting the feet and pelvis to the correct positions, which meant that, particularly at the singularity, we sometimes had issues with the knees bending backwards. We resolved this when we added the secondary task to our calculations.

3 Task Description

While the general goal was always to make Atlas jump, we went through multiple iterations of tasks, each building on our base code. This iterative approach allowed us to incrementally develop Atlas's jumping capabilities and test its performance under various conditions.

Squat

We started by programming Atlas to perform a smooth squat movement. This phase involved lowering its pelvis to a target angle by bending its knees while keeping its feet fixed on the ground.

Basic jump

After successfully implementing the squat, we incorporated the same inverse kinematics principles into a basic jump. The jump consisted of four key phases: a squat down phase (pelvis is moving downwards), a rise phase where Atlas transitions from squatting to standing (pelvis is moving upwards), a flight phase where the feet leave the ground (pelvis is moving upwards past the 1 meter it naturally reaches when Atlas stands straight), and a landing phase where Atlas returns to the starting position. We added a simulated gravity effect through an acceleration that gradually overcomes the initial vertical velocity of the jump during the flight phase and brings Atlas back to the ground. After getting the basic jump to work, we experimented with the setup to confirm that our jump was able to be performed with varied starting positions.

Frog jump

The frog jump task adds complexity by requiring Atlas to jump from a deeper squat position with its legs more spread apart that mimics a frog posture. This involved adding initial joint angles in the knees and hips. It is still able to jump off and make the landing smoothly, without any changes needed to our trajectory or kinematics calculations, as the generalized system served its purpose and was able to adjust to the new scenario.

Jump one foot off a block

Here we tested Atlas' ability to jump with one leg starting at a different elevation. By placing one foot on a raised marker block and the other on the ground, we introduced asymmetry into the jump. To achieve this, we didn't have to calculate separate trajectories for each leg, because our code was able to take the existing jump methodology and extrapolate it.

Half spin jump

The half-spin jump involved integrating rotational motion into Atlas's jump trajectory. By introducing a rotational velocity during the flight phase, Atlas performed a 180-degree turn mid-air and was still able to take off and land smoothly.

Second position pirouette

For the second position pirouette, Atlas started with its feet slightly spread and hips turned out, like the second position in ballet. From this position, it did the same rotational jump while maintaining its arms horizontal, like a dancer. This was an interesting continuation of the half-spin jump because of the outwardly turned hips. Despite the strange starting position, it was still able to squat, jump off the ground, and after spinning, land properly, without any changes to

our code, as our goal of making the algorithm sufficiently generalized to handle this type of scenario was successful.

4 Algorithm and Implementation

Initially we were deciding between utilizing center of mass calculations for our jump. If we were to use center of mass then the jump would be even more generalized and more accurate to real physics scenarios. However, we found this was too ambitious of a goal as we ended up not having time to compute all the center of mass' of every component of Atlas, so we decided to not utilize center of mass.
Despite not using center of mass to help generalize our jump, we can change the starting angles of the feet and hips and how far apart / close the legs are to each other. Also, the depth of the squat is adjustable, which can help change how high Atlas jumps.

Trajectory

In order to have Atlas perform a generalized jump, we first utilized forward kinematics to compute the position and orientation of the robot's links based on joint angles. Then, we used inverse kinematics to calculate joint angles required to achieve a desired position and orientation of the tip (pelvis). We divide the jump into different states: squatting, rising, jumping, flight, and landing. Squatting and rising have specific durations associated with these states such that, at each time step, it adjusts the robot's joint angles to simulate the motion. So, to start, for our squat down Atlas gradually lowers its pelvis height over the designated squat duration. Then Atlas transitions into its rising up state where the squat finishes and Atlas gradually raises the pelvis height to prepare for the jump, using this rising motion to push off from the ground. This rise state also has a time duration that it must follow. The next state is the jumping action, where Atlas increases its pelvis height using vertical velocity and by propelling itself from the ground. We account for acceleration due to gravity to reduce vertical velocity over time. Atlas transitions into the flight state when the vertical velocity becomes negative. This state is not tracked with time but rather with the pelvis height. When the pelvis height comes back down to 1 meter from the ground, Atlas transitions to the land state where pelvis height has returned to normal for standing straight, resets the vertical velocity, and transitions back to the squat down state.

Inverse Kinematics

Newton-Raphson Algorithm

We decided to use the Newton-Raphson algorithm because it computes the joint angles to minimize the error between the current end-effector transform and the desired target transform. The error, which combines position and orientation

differences, is mapped to joint angle updates using the Jacobian matrix. To handle redundancy or singularities, we use a damped pseudoinverse of the Jacobian to calculate the primary update, which drives the end-effector toward the target. Additionally, we implemented a secondary task to maintain natural joint configurations, such as keeping knees bent forward, by defining a secondary velocity and projecting it onto the null space of the Jacobian. This ensures that the secondary task does not interfere with the primary objective while allowing optimization of joint posture or additional constraints to prevent issues like the knees bending backwards which we encountered before adding the secondary task. The updates from the primary and secondary tasks are combined at each iteration until convergence or the maximum number of iterations is reached.

Weighted Inverse

The code uses a weighted pseudoinverse of the Jacobian, based on the damped least-squares method, to keep the inverse kinematics solution stable and reliable and handle scenarios like singularities or redundant systems. Instead of using a standard pseudoinverse, it adds a small damping factor (γ^2) to regularize the calculation, which prevents issues when the Jacobian becomes poorly conditioned or nearly singular. The weighted pseudoinverse is computed as $J^\dagger = J^T \cdot (J \cdot J^T + \gamma^2 \cdot I)^{-1}$, with I being the identity matrix. This damping not only stabilizes the updates to joint angles but also smooths out motion, especially when there are more degrees of freedom than needed. By doing this, the code ensures the inverse kinematics solution is robust, avoiding erratic or unrealistic joint movements while maintaining control. This is important for humanoid robots like Atlas, to look as realistic as possible.

Second trajectory

When we initially implemented the squat, Atlas' knees were bending inwards and outwards. As we know a squat with knees bending inwards is not realistic of a human squat. To prevent this from happening, we implemented a secondary trajectory. By projecting the secondary task into the null space of the Jacobian, it does not interfere with the primary goal of solving the inverse kinematics but allows the robot to achieve a more optimal and controlled posture even when passing through a singularity.

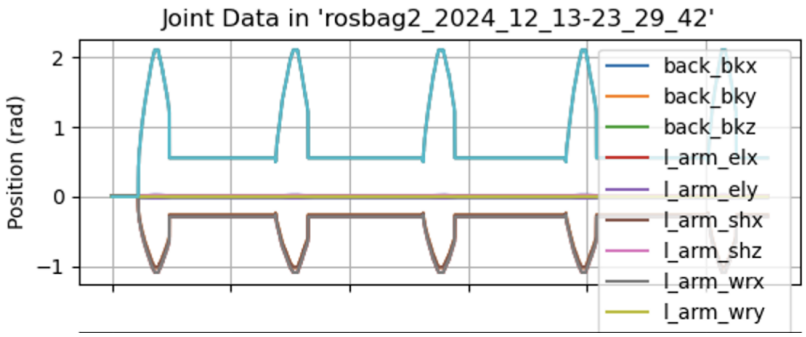
5 Particular Features

In the previous section, we discussed handling singularities with the secondary task in our inverse kinematics code. Additionally, during our implementation, we were deciding whether or not to have one leg mirror the other's trajectory. If a leg was the mirror image of another, it would lose some generalization. Thus, we decided to have two different trajectories, one for each leg. In the end, we got Atlas to perform a jump when one of its legs is at a different elevation than the other. Because we utilized two different trajectories we were able to perform

this jump while mimicking what would happen in the real world, while if we only used one trajectory and used the mirror of the trajectory we would have one of the legs weirdly hovering the ground to accommodate for one of the legs being on the marker.

6 Plot Analysis

We graphed the joint data for the basic jump. We let Atlas jump 4 times for this recording, and as you can see in the graph, the trajectories are all smooth and roughly parabolic.



7 Final Video

tinyurl.com/me133jump

8 Appendix

Atlas General Jumping File

```
import rclpy
import numpy as np
from math import pi
from trajectory_msgs.msg import JointTrajectory, JointTrajectoryPoint
from sensor_msgs.msg import JointState
from tf2_ros import TransformBroadcaster
from geometry_msgs.msg import TransformStamped

from hw5code.GeneratorNode import GeneratorNode
from hw5code.TransformHelpers import T_from_Rp, pxyz, Rotz, Transform_from_T
from hw6code.KinematicChain import KinematicChain


class Trajectory():
    def __init__(self, node):
        self.node = node
        # Create the TransformBroadcaster using the node
        self.broadcaster = TransformBroadcaster(node)

        # Set up the kinematic chain objects
        self.chain_left =
            KinematicChain(node, 'pelvis', 'l_foot', self.jointnames_left())
        self.chain_right =
            KinematicChain(node, 'pelvis', 'r_foot', self.jointnames_right())

        # Parameters for squatting and jumping
        self.squat_duration = 3 # Duration of the squat phase (seconds)
        # Duration to rise from squat to standing (seconds)
        self.rise_duration = 1.5
        # Duration to rise from squat to standing (seconds)
        self.gravity = -9.81       # Simulated gravity (m/s^2)
        self.spin_rate = np.pi  # Spin rate during the jump (radians/second)
        self.rotation_angle = 0.0   # Initial rotation angle

        # Initialize joint states for all joints
        self.joint_names = self.jointnames_all()
        self.qd = np.zeros(len(self.joint_names))   # Current joint positions
        self.qd[10] = 0.2 #lankx
        self.qd[12] = -0.2 #lhipx
        self.qd[24] = -0.2
        self.qd[26] = 0.2
```

```
        #ballet start
        #hips go out (one positive one negative)
        #feet also

        # self.qd[14] = -np.pi/2 #left hip y
        # self.qd[16] = np.pi/2 #left knee y

        self.qdot = np.zeros(len(self.joint_names)) # Current joint velocities

        # Initial joint angles for left and right legs
        self.q_init_left = np.zeros(len(self.jointnames_left()))
        self.q_init_right = np.zeros(len(self.jointnames_right()))

        # Ensure starting position is normal standing
        self.reset_to_standing_position()

        # Track state for squat/jump
        # Possible states: 'squat_down', 'rise_up', 'jump', 'flight', 'land'
        self.state = 'squat_down'
        self.timer = 0.0
        self.vertical_velocity = 0.0  # Initialize vertical velocity
        self.pelvis_height = 1.0      # Starting pelvis height

    def jointnames(self):
        # Return the full list of joint names for compatibility
        return self.jointnames_all()

    def jointnames_all(self):
        # Full list of joints as per the URDF
        return [
            'back_bkx', 'back_bky', 'back_bkz',
            'l_arm_elx', 'l_arm_ely', 'l_arm_shx', 'l_arm_shz',
            'l_arm_wrx', 'l_arm_wry', 'l_arm_wry2',
            'l_leg_akx', 'l_leg_aky', 'l_leg_hpx', 'l_leg_hpy',
            'l_leg_hpz', 'l_leg_kny',
            'neck_ry',
            'r_arm_elx', 'r_arm_ely', 'r_arm_shx', 'r_arm_shz',
            'r_arm_wrx', 'r_arm_wry', 'r_arm_wry2',
            'r_leg_akx', 'r_leg_aky', 'r_leg_hpx', 'r_leg_hpy',
            'r_leg_hpz', 'r_leg_kny'
        ]

    def jointnames_left(self):
        # Joints specific to the left leg chain
        return ['l_leg_hpz', 'l_leg_hpx', 'l_leg_hpy',
            'l_leg_kny', 'l_leg_aky', 'l_leg_akx']
```

```
def jointnames_right(self):
    # Joints specific to the right leg chain
    return ['r_leg_hpz', 'r_leg_hpx', 'r_leg_hpy',
        'r_leg_kny', 'r_leg_aky', 'r_leg_akx']

def reset_to_standing_position(self):
    # Reset joint angles to default standing position
    self.qd = np.zeros(len(self.joint_names))
    self.qdot = np.zeros(len(self.joint_names))
    self.pelvis_height = 1.0

def evaluate(self, t, dt):
    # Update state machine for jumping
    if self.state == 'squat_down':
        self.timer += dt
        self.pelvis_height = 1.0 - 0.5 * (self.timer / self.squat_duration)
        if self.timer >= self.squat_duration:
            self.state = 'rise_up'
            self.timer = 0.0

    elif self.state == 'rise_up':
        self.timer += dt
        self.pelvis_height = 0.5 + 0.25 * (self.timer / self.rise_duration)
        if self.timer >= self.rise_duration:
            self.state = 'jump'
            self.timer = 0.0
            self.vertical_velocity = 5.0  # Initial upward velocity

    elif self.state == 'jump':
        self.vertical_velocity += self.gravity * dt
        self.pelvis_height += self.vertical_velocity * dt
        self.rotation_angle += self.spin_rate * dt
        if self.vertical_velocity <= 0:
            self.state = 'flight'

    elif self.state == 'flight':
        self.vertical_velocity += self.gravity * dt
        self.pelvis_height += self.vertical_velocity * dt
        self.rotation_angle += self.spin_rate * dt
        if self.pelvis_height <= 1.0:
            self.pelvis_height = 1.0
            self.state = 'land'

    elif self.state == 'land':
        self.vertical_velocity = 0.0
```

```
            self.state = 'squat_down'
            self.timer = 0.0

        # Compute pelvis transform in world frame
        ppelvis = pxyz(0.0, 0.0, self.pelvis_height)
        Rpelvis = Rotz(self.rotation_angle)
        Tpelvis = T_from_Rp(Rpelvis, ppelvis)

        # Broadcast pelvis transform
        trans = TransformStamped()
        trans.header.stamp = self.node.get_clock().now().to_msg()
        trans.header.frame_id = 'world'
        trans.child_frame_id = 'pelvis'
        trans.transform = Transform_from_T(Tpelvis)
        self.broadcaster.sendTransform(trans)

        # Desired foot positions in the world frame (static offsets)
        left_foot_world = np.eye(4)
        left_foot_world[2, 3] = 0 #0.1 for block jump
        left_foot_world[1, 3] = 0.5

        right_foot_world = np.eye(4)
        right_foot_world[2, 3] = 0
        right_foot_world[1, 3] = -0.5 #og 0.1, changed to 0.5 for frog

        # Transform foot positions into pelvis frame
        inv_Tpelvis = np.linalg.inv(Tpelvis)
        left_foot_pelvis = inv_Tpelvis @ left_foot_world
        right_foot_pelvis = inv_Tpelvis @ right_foot_world

        # Use IK to compute joint angles for left leg
        left_joint_angles, success_left = self.chain_left.inverse_kinematics(
            left_foot_pelvis, q_init=self.q_init_left
        )
        if success_left:
           # print('left success')
            self.q_init_left = left_joint_angles
            for i, joint_name in enumerate(self.jointnames_left()):
                idx = self.joint_names.index(joint_name)
                self.qd[idx] = left_joint_angles[i]

        # Use IK to compute joint angles for right leg
        right_joint_angles, success_right =
            self.chain_right.inverse_kinematics(
                right_foot_pelvis, q_init=self.q_init_right
            )
```

```
            if success_right:
              #  print('right success')
                self.q_init_right = right_joint_angles
                for i, joint_name in enumerate(self.jointnames_right()):
                    idx = self.joint_names.index(joint_name)
                    self.qd[idx] = right_joint_angles[i]

            # Ensure non-leg joints remain static
            static_joints = set(self.jointnames_all()) -
                set(self.jointnames_left() + self.jointnames_right())
            for joint in static_joints:
                idx = self.joint_names.index(joint)
                self.qd[idx] = 0.0  # Static position

            # Publish joint states so robot_state_publisher can update all links
            js = JointState()
            js.header.stamp = self.node.get_clock().now().to_msg()
            js.name = self.joint_names
            js.position = self.qd.tolist()
            # js.velocity = self.qdot.tolist() # optional, if needed
           # self.joint_state_pub.publish(js)

            return self.qd, self.qdot


def main(args=None):
    rclpy.init(args=args)
    node = GeneratorNode('jump_generator', 100, Trajectory)
    node.spin()
    node.shutdown()
    rclpy.shutdown()


if __name__ == '__main__':
    main()
```

Kinematic Chain File

```
'''KinematicChain.py

   This is the skeleton code for Kinematic Chains (HW6 Problem 3).

   PLEASE EDIT/FIX.  See "FIXME" tags!

   chain = KinematicChain(node, basefame, tipframe, expectedjointnames)
```

```
        Initialize the kinematic chain, reading from the URDF message on
        the topic '/robot_description', sent by the robot_state_publisher.
        Determine the kinematic steps walking from the baseframe to the
        tipframe.  This expects the active joints to match the given names.

    (ptip, Rtip, Jv, Jw) = chain.fkin(q)

        Compute the forward kinematics and report the results.


    Node:         /kintest or as given
    Subscribe:    /robot_description      std_msgs/String

'''

import enum
import rclpy
import numpy as np

from rclpy.node import Node
from rclpy.qos import QoSProfile, DurabilityPolicy
from std_msgs.msg import String
from urdf_parser_py.urdf import Robot

# Grab the utilities
from hw5code.TransformHelpers import *


#
#   Single URDF Step
#
#   This captures a single step from one frame to the next.
It be of type:
#
#     FIXED     Just a fixed T-matrix shift, nothing moving, not a DOF.
#     REVOLUTE  A fixed T-matrix shift, followed by a rotation about an axis.
#     LINEAR    A fixed T-matrix shift, followed by a transation along an axis.
#
#   It contains several pieces of permanent data (coming from the URDF):
#
#     name      String showing the name
#     type      One of the above
#     Tshift    Fixed shift: Transform of this frame w.r.t. previous
#     nlocal    Joint axis (if applicable) in the local frame
#
```

```
#   We also add information how this relates to the active joints:
#
#     dof         If an active dof (not FIXED), the dof number
#

# Define the joint types.
class JointType(enum.Enum):
    FIXED = 0
    REVOLUTE = 1
    LINEAR = 2


# Define a single step in the URDF (kinematic chain).
class URDFStep():
    def __init__(self, name, type, Tshift, nlocal):
        # Store the permanent/fixed/URDF data.
        self.name = name  # Joint name
        self.type = type  # Joint type (per above enumeration)
        self.Tshift = Tshift  # Transform w.r.t. previous frame
        self.nlocal = nlocal  # Joint axis in the local frame

        # Match against the joint numbers
        self.dof = None  # Joint DOF number (or None if FIXED)


#
#   Kinematic Chain Object
#
#   This stores the information provided by the URDF in the form of
#   steps (see above).  In particular, see the fkin() function, as it
#   walks up the chain to determine the transforms.
#

# Define the full kinematic chain
class KinematicChain():
    # Helper functions for printing info and errors.
    def info(self, string):
        self.node.get_logger().info("KinematicChain:" + string)

    def error(self, string):
        self.node.get_logger().error("KinematicChain:" + string)
        raise Exception(string)

    # Initialization - load the URDF!
    def __init__(self, node, baseframe, tipframe, expectedjointnames):
        # Store the node (for the helper functions).
```

```
        self.node = node

        # Store joint names for reference in IK.
        self.joint_names = list(expectedjointnames)

        # Prepare the list of steps.
        self.steps = []
        self.dofs = 0

        # Create a temporary subscriber to receive the URDF.
We use
        # the TRANSIENT_LOCAL durability, so that we see the last
        # message already published (if any).
        self.info("Waiting for the URDF to be published...")
        self.urdf = None

        def cb(msg):
            self.urdf = msg.data

        topic = '/robot_description'
        quality = QoSProfile(durability=DurabilityPolicy.TRANSIENT_LOCAL,
                             depth=1)
        sub = self.node.create_subscription(String, topic, cb, quality)
        while self.urdf is None:
            rclpy.spin_once(self.node)
        self.node.destroy_subscription(sub)

        # Convert the URDF string into a Robot object and report.
        robot = Robot.from_xml_string(self.urdf)
        self.info("Proccessing URDF for robot '%s'" % robot.name)

        # Parse the Robot object into a list of URDF steps from the
        # base frame to the tip frame. Search backwards, as the robot
        # could be a tree structure.
        frame = tipframe
        while frame != baseframe:
            # Look for the URDF joint to the parent frame.
            joint = next((j for j in robot.joints if j.child == frame), None)
            if joint is None:
                self.error(f"Unable to find joint connecting to '{frame}'")
            if joint.parent == frame:
                self.error(f"Joint '{joint.name}' connects '{frame}' to itself")
            frame = joint.parent

            # Check the type (use the above enumeration)
            if joint.type in ['revolute', 'continuous']:
```

```
                type = JointType.REVOLUTE
            elif joint.type == 'prismatic':
                type = JointType.LINEAR
            elif joint.type == 'fixed':
                type = JointType.FIXED
            else:
                self.error(f"Joint '{joint.name}' has unknown type '{joint.type}

            # Normalize the axis for REVOLUTE or LINEAR joints.
            nlocal = None
            if type != JointType.FIXED:
                nlocal = n_from_URDF_axis(joint.axis)
                nlocal /= np.linalg.norm(nlocal)

            # Create the URDF step.
            self.steps.insert(0, URDFStep(
                name=joint.name,
                type=type,
                Tshift=T_from_URDF_origin(joint.origin),
                nlocal=nlocal
            ))

        # Assign DOF numbers and count DOFs.
        for step in self.steps:
            if step.type != JointType.FIXED:
                step.dof = self.dofs
                self.dofs += 1
            else:
                step.dof = None

        # Confirm the active joint names match the expectation.
        jointnames = [s.name for s in self.steps if s.type != JointType.FIXED]
        if jointnames != self.joint_names:
            self.error(f"Chain does not match expected joint names:"
                       f" {expectedjointnames}")

    # Compute the forward kinematics!
    def fkin(self, q):
        # Check the number of joints
        if (len(q) != self.dofs):
            self.error("Number of joint angles (%d) does not chain (%d)" %
                       (len(q), self.dofs))

        # We will build up three lists.  For each non-fixed (real)
        # joint remember the type, position, axis w.r.t. world.
        type = []
```

```
p = []
n = []

# Initialize the T matrix to walk up the chain, w.r.t. world frame!
T = np.eye(4)

# Walk the chain, one URDF step at a time, adjusting T as we
# go.  This could be a fixed or active joint.
for step in self.steps:
    '''
    Note the step is of type URDFStep and contains:
    step.Tshift     Transform w.r.t. the previous frame
    step.nlocal     Joint axis in the local frame
    step.dof        Joint number
    which also tell us
    q[step.dof]     Joint position (angle or displacement)'''

    # Take action based on the joint type: Move the transform
    # T up the kinematic chain (remaining w.r.t. world frame).
    if step.type is JointType.REVOLUTE:
        # Revolute is a shift followed by a rotation:
        T = T @ step.Tshift @ T_from_Rp(Rotn(step.nlocal, q[step.dof]),
                                        pzero())
    elif step.type is JointType.LINEAR:
        # Linear is a shift followed by a translation:
        T = T @ step.Tshift @ T_from_Rp(Reye(),
                                        step.nlocal * q[step.dof])
    else:
        # Fixed is only a shift.
        T = T @ step.Tshift

    # For active joints, store the type, positon and axis info
    # (w.r.t. world frame).
    if step.type != JointType.FIXED:
        type.append(step.type)
        p.append(p_from_T(T))
        n.append(R_from_T(T) @ step.nlocal)

# Collect the tip information.
ptip = p_from_T(T)
Rtip = R_from_T(T)

# Collect the jacobian for each active joint.
Jv = np.zeros((3, self.dofs))
Jw = np.zeros((3, self.dofs))
for i in range(self.dofs):
```

```
            '''FIXME AGAIN
            After we walked the chain above, we now have:
            type[i]    Joint type
            p[i]       Joint position w.r.t. world
            n[i]       Joint axis w.r.t. world'''

            # Fill in the appropriate Jacobian column based on the
            # type.  The Jacobian (like the data) is w.r.t. world.
            if type[i] is JointType.REVOLUTE:
                # Revolute is a rotation:
                Jv[:, i] = cross(n[i], ptip - p[i])
                Jw[:, i] = n[i]
            elif type[i] is JointType.LINEAR:
                # Linear is a translation:
                Jv[:, i] = n[i]
                Jw[:, i] = np.zeros(3)

        # Return the info
        return (ptip, Rtip, Jv, Jw)

    def inverse_kinematics(self, target_T, q_init=None, max_iters=100,
                           tol=1e-3):
        """
        Solve inverse kinematics with a secondary task to constrain knee motion.

        Args:
            target_T (numpy.ndarray): Desired 4x4 transform of the tip frame
            w.r.t. base frame.
            q_init (numpy.ndarray): Initial guess for joint angles (optional).
            max_iters (int): Maximum number of iterations.
            tol (float): Convergence tolerance for the position error.

        Returns:
            (numpy.ndarray, bool): Joint angles and success flag.
        """
        # Initialize joint angles
        q = np.zeros(self.dofs) if q_init is None else np.copy(q_init)
        # q = (0, 0, -1/2.0, 1, -1/2.0, 0)
        for iteration in range(max_iters):
            # Compute current forward kinematics

            ptip, Rtip, Jv, Jw = self.fkin(q)

            # Compute position error
            p_target = target_T[:3, 3]
            dp = p_target - ptip
```

```
        R_target = target_T[:3, :3]

        # Compute orientation error
        orientation_error = (np.cross(Rtip[:, 0], R_target[:, 0]) + \
                             np.cross(Rtip[:, 1], R_target[:, 1]) + \
                             np.cross(Rtip[:, 2], R_target[:, 2])) / 2

        # Combine errors
        error = np.hstack((dp, orientation_error))

        if np.linalg.norm(error) < tol:
            return q, True

        # Compute the full Jacobian (stack Jv and Jw)
        J = np.vstack((Jv, Jw))
        gamma = 0.05
        # Primary task: Solve for joint updates using pseudoinverse
        J_pinv = J.T @ np.linalg.pinv(J @ J.T + gamma**2 * np.eye(6))
        dq_primary = J_pinv @ error

        # Secondary task: Keep knees bending forward
        knee_indices = [self.joint_names.index(
            'l_leg_kny') if 'l_leg_kny' in self.joint_names else None,
                        self.joint_names.index(
                            'r_leg_kny') if 'r_leg_kny' in self.joint_names
                        else None]

        q_target = np.copy(q)
        for idx in knee_indices:
            if idx is not None:
                q_target[idx] = np.radians(45) # Desired forward knee angle
        secondary_task_velocity = -0.10 * (q - q_target)

        # Apply null-space projection for the secondary task
        null_space_projection = np.eye(len(q)) - J_pinv @ J
        dq_secondary = null_space_projection @ secondary_task_velocity

        # Combine primary and secondary tasks
        dq = dq_primary + dq_secondary

        # Update joint angles
        q += dq

    # If we exit the loop, IK failed to converge

    return q, False
```